

Project Title: SLAIT – Secure Language Assembly Inspector Tool

Team Members:

- Maria Linkins-Nielsen – mlinkinsniel2022@my.fit.edu
- Michael Bratcher – mbratcher2021@my.fit.edu

Faculty Advisor: Dr. Marius Silaghi – msilaghi@fit.edu

Client: Dr. Marius Silaghi – CSE Professor & Faculty Advisor

Progress of Current Milestone:

Task	Completion%	Michael	Maria	Reasoning/To-Do
1.Connect Frontend HTTP Post to Backend Execution	100%	50%	50%	The Project was successful in sending and returning results through a development Ubuntu Apache server.
2. End-to-End Testing	75%	50%	50%	The user end flow was successfully achieved, but small changes will be made regarding JSON that need to be retested.
3. Structured Error Handling & User Feedback	50%	25%	75%	Structured backend JSON implemented to report execution failures such as missing input files, pipeline errors, and container issues. Remaining work includes refining error categorization and integrating user-facing error display within the frontend interface.

4. System Documentation & Demo Preparation	50%	50%	50%	Documentation still needs to be drafted, but a basic entire system demo exists and just needs polish, aside from server snapshot footage.
--	-----	-----	-----	---

Discussion of Milestone Tasks:

1. A personal server was utilized to replicate the environment anticipated for the production server, specifically an Ubuntu server leveraging Apache2 to serve the frontend on port 80. As previously noted, Nginx was the original intended web server; however, given that the production environment only supports Apache, the frontend-backend communication strategy was revised accordingly. This revised approach employs a Flask script paired with a background system process to facilitate communication between the two layers. Both the frontend and backend repositories are cloned onto the server, after which the frontend is compiled and served via Apache. The backend Docker image is then built, and the Flask script, which is responsible for relaying JSON between the frontend and the containerized backend, is configured as a system service and set to start automatically.

2. Once the frontend-backend communication flow was successfully established, end-to-end testing was conducted through the webpage to verify that the minimum viable product met all defined requirements. The complete project flow test confirmed that the application supports the following functionality: selecting or composing a file within the application, with templates available to assist the user; selecting specific registers and flags on designated lines to monitor their values; formatting and transmitting this data to a containerized backend environment; executing the submitted files and recording register values according to the provided instructions; returning the requested information to the frontend; and formatting the results into both graphical and tabular representations. This flow operates as a continuous cycle, allowing the process to be repeated multiple times without requiring a page refresh or manual reset by the user. Additionally, all functionality is contained within a single webpage, meaning only the relevant output widgets are updated upon receiving a response, rather than requiring a full page reload.

You can find the demo for this here: [🔴 SLAIT Demo Screenrecord.mov](#)

3. Structured error handling was introduced on the backend to improve how execution failures are detected and communicated to the frontend. Previously, errors such as missing input files, assembly compilation errors, and container execution issues were only visible through terminal output, which limited usability during testing. The backend execution script was updated to return structured JSON responses describing successful execution results and failure conditions. This supports clearer user-facing feedback. Additional work remains to expand error categorization and integrate these responses into visible frontend notifications for users.

4. Documentation for setting up the system on a server has been created, but user guides are not yet drafted. A basic full function demo is recorded for the user end, but we'd like to edit and formalize the video into something more than just a screen recording.

Contribution to Milestone Tasks:

Michael:

I worked on the development server environment, setting up both the frontend and backend to the environment, creating the flask script, creating the system service, and making documentation on the steps for recreation.

Maria:

I focused on improving the reliability of the containerized analysis pipeline. I worked on the structured JSON output for frontend visualization for error handling purposes.

1. Plan for the next Milestone (task matrix) or
[skip if this is for Milestone 6]

Task	Michael	Maria
Deploy Build to Production Server	50%	50%

○ Task 3:

○ Task 4:

○ Task 5:

7. Faculty Advisor Signature: _____ Date: _____

8. Evaluation by Faculty Advisor

- Faculty Advisor: detach and return this page to Dr. Chan (HC 209) or email the scores to pkc@cs.fit.edu
- Score (0-10) for each member: circle a score (or circle two adjacent scores for .25 or write down a real number between 0 and 10)

Michael	0	1	2	3	4	5	5.5	6	6.5	7	7.5	8	8.5	9	9.5	10
---------	---	---	---	---	---	---	-----	---	-----	---	-----	---	-----	---	-----	----

Maria	0	1	2	3	4	5	5.5	6	6.5	7	7.5	8	8.5	9	9.5	10
-------	---	---	---	---	---	---	-----	---	-----	---	-----	---	-----	---	-----	----

○ Faculty Advisor Signature: _____ Date:
